

10-Deployment-Infrastructure-Architecture.md

Healthcare Clinical Intelligence Platform

Production Infrastructure Architecture

Version: 1.0

Status: Approved Design

Mission

Build an infrastructure platform that is:

- Production Ready
- Highly Available
- Secure
- Cost Efficient
- Scalable
- Fault Tolerant
- Compliance Ready

Target:

10,000+ Queries / Day
100,000+ Documents
10M+ Chunks
99.9% Availability

Infrastructure Philosophy

Development

Local Docker

Staging

Single VPS

Production

AWS + Kubernetes

High-Level Architecture

```
graph TD
    Users --> Cloudflare
    Cloudflare --> AWS_Load_Balancer[AWS Load Balancer]
    AWS_Load_Balancer --> Kubernetes_Cluster[Kubernetes Cluster]
    Kubernetes_Cluster -.-> Frontend
    Kubernetes_Cluster -.-> Backend
    Kubernetes_Cluster -.-> Workers
    Frontend -.-> Redis
    Frontend -.-> Qdrant
    Backend -.-> Neo4j
    Workers -.-> Postgres
```

Environment Strategy

Local

Purpose:

Developer Productivity

Staging

Purpose:

Testing

Production

Purpose:

Customer Traffic

Frontend Infrastructure

Technology:

- NextJS

Deployment:

- Vercel (initial)
 - Kubernetes (later)
-

Backend Infrastructure

Technology:

- FastAPI

Deployment:

- Docker
 - Kubernetes
-

LangGraph Infrastructure

Separate Service

Agent Runtime Cluster

Responsibilities:

- Agent Execution
- Tool Calling
- Workflow Orchestration

Ingestion Infrastructure

Separate Worker Pool

Parser Workers

Embedding Workers

Graph Workers

Validation Workers

Never run on API servers.

Queue Infrastructure

Phase 1

Redis + Celery

Phase 2

Temporal

Database Architecture

Supabase

Stores:

Users

Organizations

Chats

Memory

Audit Logs

Evaluations

Production Recommendation

Managed Postgres

Separate Instance

Qdrant Architecture

Purpose:

Vector Retrieval

Collections

medical_text

medical_tables

medical_images

external_sources

semantic_cache

Scaling

Horizontal

Replication

Enabled

Neo4j Architecture

Purpose:

GraphRAG

Knowledge Graph

Deployment

Dedicated Node

Backup

Daily

Redis Architecture

Purpose:

Caching

Queues

Sessions

Memory

Cache Layers

Query Cache

Retrieval Cache

Semantic Cache

Embedding Cache

Object Storage

AWS S3

Stores:

Raw Files

Versions

Images

Tables

Artifacts

Exports

CDN Layer

Cloudflare

Responsibilities:

Caching

DDoS Protection

WAF

SSL

Rate Limiting

Kubernetes Architecture

Namespaces

```
frontend  
backend  
agents  
workers  
monitoring  
data-services
```

Backend Pods

FastAPI Pods

Autoscaling

CPU

Memory

Request Volume

Agent Pods

Planner

Retriever

Validator

Safety

Response

Scaling

Independent

Worker Pods

Parser

Embedding

Graph

Validation

Indexing

Observability Stack

Langfuse

AI Observability

OpenTelemetry

Tracing

Prometheus

Metrics

Grafana

Dashboards

Loki

Logs

CI/CD Pipeline

GitHub

↓

GitHub Actions

↓

Tests

↓

Security Checks

↓

Deploy

Deployment Pipeline

Code Push

↓

Tests

↓

Security Scan

↓

Build Docker Images

↓

Deploy Staging

↓

Approval

↓

Production

Secrets Management

Never store secrets in code.

Use:

AWS Secrets Manager

Stores:

API Keys

Database Credentials

JWT Secrets

Provider Credentials

Security Architecture

Authentication

Supabase Auth

Authorization

RBAC

Department Permissions

Tenant Isolation

Encryption

At Rest

AES-256

In Transit

TLS

Compliance Architecture

Supports:

HIPAA

GDPR

SOC2

ISO27001

Backup Strategy

PostgreSQL

Daily

Qdrant

Daily Snapshots

Neo4j

Daily Backup

S3

Versioning Enabled

Disaster Recovery

Target:

RPO < 1 Hour

RT0 < 4 Hours

Multi-Tenant Isolation

Every query filtered by:

organization_id

Mandatory.

Rate Limiting

Per User

Per API Key

Per Organization

API Gateway

Responsibilities:

Authentication

Rate Limiting

Logging

Metrics

Cost Optimization Strategy

Rule 1

Use Cheap Models First

Rule 2

Aggressive Caching

Rule 3

Incremental Reindexing

Rule 4

Adaptive Retrieval

Rule 5

Scale Workers Independently

Performance Targets

API

<300ms

Retrieval

<1.5s

Agent Workflow

<3s

Full Response

<5s

Infrastructure Monitoring

Track:

CPU

Memory

Network

Storage

Costs

Latency

Errors

Availability

Production Readiness Checklist

Multi-Tenant

RBAC

Caching

Observability

Backups

Disaster Recovery

CI/CD

Autoscaling

Rate Limiting

Audit Logging

Security

Compliance

Future Scaling

Phase 2

Multi-AZ

Phase 3

Multi-Region

Phase 4

Global Healthcare Platform

Success Targets

99.9% Uptime

<5s Responses

<1% Failure Rate

10,000+ Queries/Day

100,000+ Documents

10M+ Chunks

Final Principle

Infrastructure should never be the bottleneck.

The platform must scale, recover, observe, secure, and govern itself while maintaining healthcare-grade reliability.